Basic JavaScript Concepts and Functions Q&A

🧠 1. Basic JavaScript Questions

Q1. What is JavaScript? How is it different from Java?

> **JavaScript is a lightweight, interpreted programming language** used to make web pages interactive (client-side scripting).
>
> **Differences from Java:**
>
> - **JavaScript** is **interpreted** (JIT compiled), dynamically typed, and primarily focuses on the browser/web environment.
> - **Java** is a **compiled**, statically typed, object-oriented language used for general-purpose applications (runs on the Java Virtual Machine - JVM).

Q2. Explain 'var', 'let', and 'const' keywords.

> - **var**: **Function-scoped**. Can be redeclared and updated. It is generally avoided in modern JS due to hoisting issues.
> - **let**: **Block-scoped** (defined within `{}`). Can be updated but not redeclared in the same scope.
> - **const**: **Block-scoped**. Cannot be redeclared or updated after initial assignment. Must be initialized at declaration.

Q3. What are JavaScript data types?

> JavaScript has two main categories of data types:
>
> - **Primitive Types** (values are immutable): **String, Number, Boolean, Undefined, Null, Symbol, BigInt.**
> - **Non-primitive Types** (values are mutable and stored by reference): **Object, Array, Function.**

Q4. What are closures in JavaScript?

A **closure is the combination of a function and the lexical environment within which that function was declared**. It means an inner function retains access to variables from its outer (enclosing) function, **even after the outer function has finished executing**.

Example:

```
function outer() {
  let count = 0; // The inner function closes over this variable
  return function inner() {
    count++;
    console.log(count);
  };
}
const counter = outer();
counter(); // 1
counter(); // 2
```

Q5. Explain the concept of hoisting.

Hoisting is JavaScript's behavior where **variable and function declarations are moved (conceptually) to the top of their current scope** during the compilation phase, regardless of where the declarations are written.

- Only declarations are hoisted, not initialization/assignment.
- `var` variables are hoisted and initialized to `undefined`.
- `let` and `const` variables are hoisted but not initialized, resulting in a "Temporal Dead Zone" if accessed before declaration.

Example (using var):

```
console.log(a); // undefined (declaration is hoisted)
var a = 10;
// Actual execution:
// var a;
// console.log(a);
// a = 10;
```

Q6. What are template literals in ES6?

Template literals (Template Strings) are a feature introduced in ES6 that allow for **easier string manipulation**, including multiline strings and **string interpolation** using embedded expressions.

They are created using **backticks (`` ` ` ``)**.

Example:

```
let name = "Jagdeep";
console.log(`Hello, ${name}!`); // Output: Hello, Jagdeep!
```

Q7. Difference between == and ===:

- **==** (Loose Equality): Checks for **value only**. It performs type coercion (converts types) before comparison.

  ```
  console.log(5 == '5'); // true
  ```

- **===** (Strict Equality): Checks for **value AND type**. It does not perform any type coercion. This is generally preferred for predictability.

  ```
  console.log(5 === '5'); // false
  ```

Q8. What is an IIFE?

An **IIFE (Immediately Invoked Function Expression)** is a JavaScript function that **runs as soon as it is defined**. It creates a local scope, which prevents variables inside it from polluting the global namespace.

Syntax:

```
(function () {
  console.log("Runs immediately");
})();
```

Q9. What is NaN in JavaScript?

NaN stands for "**Not-a-Number**." It is a special numeric value that represents the result of an **invalid or unrepresentable numeric operation** (e.g., `0 / 0` or `'hello' * 5`).

Crucially, `NaN` is not strictly equal to itself (`NaN === NaN` is `false`).

To check for `NaN`, use: **`isNaN(value)`** or `Number.isNaN(value)`.

⚙️ 2. Functions and Scope

Q1. Function Declaration vs Function Expression

- **Declaration**: Defined as a separate statement (e.g., `function declared() {}`). It is **fully hoisted**, meaning it can be called before its definition in the code.
- **Expression**: Defined as part of a larger expression, often assigned to a variable (e.g., `const expressed = function () {};`). It is **not hoisted** in the same way; only the variable name is hoisted. It cannot be called before the assignment line.

Q2. Arrow Functions

Arrow Functions (introduced in ES6) provide a concise syntax for writing functions. They differ from regular functions in a few key ways:

- **Shorter Syntax**: Implicit `return` for single expressions.
- **No Binding of `this`**: They **do not have their own `this` value**; they inherit the `this` value from the enclosing lexical scope (their parent).
- **No `arguments`, `super`, or `new.target`.**

Example:

```
const sum = (a, b) => a + b;
```

Q3. Scope Types:

Scope determines the accessibility of variables and functions at different parts of the code.

- **Global Scope**: Variables defined outside any function or block. Accessible everywhere in the application.
- **Function Scope**: Variables declared using `var` inside a function. Accessible only within that function.
- **Block Scope**: Variables declared using **`let` or `const`** inside any curly braces (`{}`), such as loops or conditional statements.

Q4. 'this' keyword:

The **`this` keyword** refers to the **object that owns the current execution context** (the current code being executed). Its value is determined dynamically when a function is called.

- In a method, `this` refers to the owner object.
- In a simple function call (non-strict mode), `this` defaults to the global object (`window` or `global`).
- **In Arrow Functions**: Arrow functions **do not bind their own `this`**; they lexically inherit `this` from the scope in which they are defined.

Q5. Currying:

Currying is a functional programming technique where a function that takes **multiple arguments is transformed into a sequence of functions**, each taking a single argument.

Example:

```
// Normal function: add(2, 3)
const add = a => b => a + b;

console.log(add(2)(3)); // 5
// add(2) returns a new function (b => 2 + b)
// This new function is then called with (3)
```

🧱 3. Object-Oriented JavaScript

Q1. What is prototypal inheritance in JavaScript?

Prototypal inheritance is the mechanism where **objects inherit properties and methods from other objects** (the prototype object), rather than from classes.

Every JavaScript object has an internal link (known as `[[Prototype]]` or accessed via `__proto__` or `Object.getPrototypeOf()`) to another object, and when a property is accessed, the JavaScript engine traverses this link chain until the property is found.

Example:

```
const person = {
  greet() {
    console.log("Hello");
  },
};
const student = Object.create(person); // student's prototype is person
student.greet(); // Output: Hello (Method inherited from person)
```

Q2. How can you create an object in JavaScript? Provide an example.

There are several ways to create an object:

- **Object Literal (Recommended)**:

```
const car = { brand: "Toyota", model: "Camry" };
```

- **Using `new Object()`**:

```
const user = new Object();
user.name = "Jagdeep";
```

- **Using Constructor Function (Pre-ES6)**:

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}
const p1 = new Person("Jagdeep", 22);
```

- **Using `Object.create()`**:

```
const animal = { type: "mammal" };
const dog = Object.create(animal); // dog inherits from animal
```

- **Using ES6 Classes**:

```
class Vehicle {}
const v1 = new Vehicle();
```

Q3. What is the difference between Object.create() and new in JavaScript?

Feature		
Purpose	Creates a new object and **manually sets its prototype** to an existing object.	Creates a new object instance and **sets its prototype to the constructor's `.prototype` property**.
Constructor Call	**Does not call a constructor function**.	**Calls the constructor function** to initialize instance properties (`this` binding).
Usage	Used for implementing prototypal inheritance directly.	Used for instantiating objects from Constructor Functions or ES6 Classes.

Q4. What are getters and setters in JavaScript? Give examples.

Getters and Setters (accessors) are methods used to get or set a property value in an object, but they look like normal properties when accessed externally.

- **Getter (`get`)**: A method that **gets the value of a property**. It allows computation on access.
- **Setter (`set`)**: A method that **sets the value of a property**. It allows validation or modification of the input value before storing it.

Example:

```
const user = {
  firstName: "Jagdeep",
  lastName: "Singh",
  get fullName() { // Getter
    return `${this.firstName} ${this.lastName}`;
  },
  set fullName(name) { // Setter
    [this.firstName, this.lastName] = name.split(" ");
  },
```

```
};
console.log(user.fullName); // Calls the getter: "Jagdeep Singh"
user.fullName = "Aman Gupta"; // Calls the setter
console.log(user.firstName);  // "Aman"
```

Q5. Explain the concept of the 'constructor' in JavaScript.

A **constructor** is a special function or method used to **create and initialize an object** created from a class or a constructor function. It sets up the initial state of the object.

- **Constructor Function (Pre-ES6)**: It's a regular function invoked using the `new` **keyword**. Inside the function, `this` refers to the new object instance being created.
- **ES6 Classes**: It's a specific method named `constructor`. It is called automatically when an object is instantiated.

Example (ES6 Class):

```
class Car {
  constructor(brand, model) { // The constructor method
    this.brand = brand;
    this.model = model;
  }
}
const car1 = new Car("Honda", "City");
```

Q6. What is the purpose of the 'super' keyword in ES6 classes?

The `super` **keyword** is used inside a subclass (child class) to **access and call functions belonging to its parent class** (superclass).

- **In the Constructor**: `super()` is used to call the parent's constructor. This must be done before accessing `this` in the child constructor.
- **In Other Methods**: `super.methodName()` is used to call a method defined on the parent class.

Example:

```
class Animal {
  constructor(name) {
    this.name = name;
```

```
  }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name); // Calls Animal's constructor
    this.breed = breed;
  }
}
```

---

4. Asynchronous JavaScript

Q1. What is the event loop in JavaScript? How does it work?

The **Event Loop** is a fundamental mechanism in JavaScript's runtime environment that enables non-blocking asynchronous operations, despite JS being single-threaded. It continuously checks the Call Stack and the Callback Queue.

Its job is simple: **If the Call Stack is empty, it pushes the next function from the Callback Queue onto the stack for execution.**

Simplified Flow:

1. Synchronous code runs in the **Call Stack**.
2. Asynchronous operations (e.g., `setTimeout`, API calls) are offloaded to the **Web APIs** environment.
3. When the async task completes, its callback function is moved to the **Callback Queue** (Macro/Microtask).
4. The Event Loop picks up the callback from the queue only when the Call Stack is idle.

Q2. Explain promises in JavaScript. How do you handle errors in promises?

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation and its resulting value. It allows for cleaner code chaining than traditional callbacks.

A promise exists in one of three states:

- **Pending**: Initial state, neither fulfilled nor rejected.
- **Fulfilled**: The operation completed successfully (calling `resolve`).

- **Rejected**: The operation failed (calling `reject`).

Error Handling:

Errors in a promise chain are handled using the **`.catch()` method**, which is essentially a shorthand for `.then(null, rejectionHandler)`.

```
promise
  .then(result => console.log(result)) // Handles fulfillment
  .catch(error => console.log(error)); // Handles rejection (errors)
```

Q3. What are async/await in JavaScript? Provide an example.

`async/await` is syntactical sugar built on top of Promises, making asynchronous code look and behave more like synchronous code, thus improving readability.

- **`async`**: Used to declare a function as asynchronous. An async function **always returns a Promise**.
- **`await`**: Used inside an `async` function. It pauses the execution of the async function until the Promise it precedes is resolved.

Example:

```
async function fetchData() {
  try {
    // Execution pauses here until the promise resolves
    const res = await fetch("https://api.example.com/data");
    const data = await res.json();
    console.log(data);
  } catch (error) {
    // Standard try/catch block handles promise rejection/errors
    console.log("Error:", error);
  }
}
fetchData();
```

Q4. What is the difference between a callback and a promise?

Feature		
Definition	A function passed as an argument to be executed later.	An object representing the future completion/failure of an async operation.
Drawback	Can lead to **"callback hell"** (deeply nested, hard-to-read code).	Avoids nesting; makes chaining operations easier with `.then()`.
Error Handling	Must be handled manually (by convention, passing error as first argument).	Handled uniformly with the dedicated `.catch()` **method**.

Q5. How does setTimeout and setInterval work in JavaScript?

Both functions are part of the Web API environment and are used to schedule function execution asynchronously.

- **setTimeout(func, delay)**: Executes a function **only once** after a specified delay (in milliseconds).
- **setInterval(func, delay)**: Executes a function **repeatedly**, starting after the initial delay, and then repeatedly at intervals of that delay.

Crucially, neither guarantees exact timing; they only guarantee the function is added to the Macrotask Queue *after* the delay, waiting for the Call Stack to clear.

Example:

```
setTimeout(() => console.log("Runs once after 2 sec"), 2000);
// Returns an ID that can be used with clearTimeout()
const intervalID = setInterval(() => console.log("Runs every 3 sec"), 30
// To stop: clearInterval(intervalID);
```

Q6. What are microtasks and macrotasks in the JavaScript event loop?

These are two different queues for handling asynchronous callbacks, and the Event Loop prioritizes them differently.

Type	Examples	Priority
Microtasks	Promises (`.then()`, `.catch()`), `MutationObserver`.	**Higher** (The event loop processes all microtasks before moving to the next macrotask).
Macrotasks	`setTimeout`, `setInterval`, I/O, UI rendering.	Lower (The event loop processes one macrotask per cycle).

Execution Order:

```
// 1. Synchronous Code
console.log("Main task");
// 2. Microtasks
Promise.resolve().then(() => console.log("Microtask"));
// 3. Macrotasks
setTimeout(() => console.log("Macrotask"), 0);

// Output Sequence (guaranteed):
// 1. Main task
// 2. Microtask
// 3. Macrotask
```

🌳 5. DOM Manipulation

Q1. What is the DOM? How does JavaScript interact with it?

The **DOM (Document Object Model)** is a programming interface for web documents. It represents the structure of a web page as a **tree of objects**, where each node is an object representing a part of the document (elements, attributes, text).

JavaScript interacts with the DOM by treating the entire document as a hierarchical structure. It uses the methods and properties exposed by the DOM to **dynamically access, modify, and update** the content, style, and structure of the webpage.

Example:

```
document.getElementById("title").textContent = "Hello, World!";
```

```
// Finds the element and changes its content.
```

Q2. How do you select an HTML element in JavaScript?

Elements can be selected using various methods available on the `document` object:

- `document.getElementById("id")`: Selects the single element with the matching ID.
- `document.getElementsByClassName("class")`: Selects all elements with the matching class (returns a live HTML Collection).
- `document.getElementsByTagName("div")`: Selects all elements with the matching tag name (returns a live HTML Collection).
- **`document.querySelector(".class")`**: Selects the **first** element that matches the specified CSS selector.
- **`document.querySelectorAll(".class")`**: Selects **all** elements that match the specified CSS selector (returns a static NodeList).

Q3. Explain event delegation in JavaScript.

Event delegation is a technique where you place a **single event listener on a stable parent element** instead of attaching individual listeners to numerous child elements.

It works because of **event bubbling**—when an event is triggered on a child element, it "bubbles up" the DOM tree to its parents. The parent's listener captures the event and uses `event.target` to identify which specific child element was clicked.

Benefits:

- **Memory efficiency**: Only one listener is created, saving resources.
- **Handles dynamic elements**: Automatically works for new elements added to the parent later.

Example:

```
document.getElementById("list").addEventListener("click", function(e) {
  // Check if the click originated from an LI element
  if (e.target.tagName === "LI") {
    alert("You clicked on " + e.target.textContent);
  }
});
```

Q4. What is the difference between 'addEventListener' and 'onclick'?

Feature		
Handlers	Can only attach **one handler** per event type (the last one overrides previous ones).	Can add **multiple handlers** to the same event type on the same element.
Removal	Requires setting the property to `null` (e.g., `element.onclick = null`).	Requires `removeEventListener()`, needing a reference to the original function.
Phase	Only supports the bubbling phase.	Supports both the **capturing and bubbling phases** (via a third argument).

`addEventListener()` is the modern, preferred method.

Q5. How can you prevent event bubbling in JavaScript?

To stop an event from propagating (bubbling up) to its parent elements after it has been handled, you use the **`event.stopPropagation()` method** inside the event listener.

Example:

```
document.getElementById("child").addEventListener("click", (e) => {
  e.stopPropagation(); // Stops the event here
  console.log("Child clicked");
});
// The parent listener above this element will NOT fire.
```

Q6. How do you dynamically create and append DOM elements using JavaScript?

You use the `document.createElement()` method to create a new element node, set its properties (content, class, style), and then use an `appendChild()` or `insertBefore()` method on a parent element to place it in the DOM.

Creation and Insertion Example:

```
// 1. Create element
const newDiv = document.createElement("div");

// 2. Set properties
newDiv.textContent = "New element added!";
newDiv.className = "dynamic-content";

// 3. Append to a parent (e.g., the body)
document.body.appendChild(newDiv);
```

You can remove the element later using `newDiv.remove()`.

⚙️ 6. Advanced JavaScript

Q1. What are JavaScript modules? How do you export and import modules in ES6?

JavaScript Modules are files designed to encapsulate code, allowing you to split large applications into **reusable, self-contained files**. Modules ensure that variables and functions defined inside are private by default (scoped only to the module).

In ES6, this is managed using the `export` and `import` keywords:

- **`export`**: Exposes variables, functions, or classes for use outside the current module.
- **`import`**: Allows code from another module to be brought into the current file.

Example:

```
// math.js
export function add(a, b) { return a + b; }
export const PI = 3.14;

// app.js
import { add, PI } from './math.js';
console.log(add(2, 3)); // 5
```

Q2. Explain the difference between synchronous and asynchronous JavaScript.

The difference lies in how long operations take and whether they block the main execution thread.

Type	Description	Example Flow
Synchronous	Code executes **line by line in sequence**. The execution of a line must be completed before the next line starts. This is "blocking."	`console.log("One");` (wait) → `console.log("Two");`
Asynchronous	Code execution is offloaded and **runs independently of the main thread**. The thread continues to execute the next line while waiting for the result. This is "non-blocking."	`setTimeout(callback, 1000);` (offloaded) → `console.log("Next line");` → Callback runs later.

Q3. What are higher-order functions in JavaScript? Provide examples.

A **higher-order function (HOF)** is a function that does one or both of the following:

1. Takes one or more functions as **arguments** (a callback function).
2. **Returns** a function as its result.

Common Examples (Built-in HOFs):

- Array methods like **`map()`**, **`filter()`**, **and** **`reduce()`**.
- Time functions like `setTimeout()` and `setInterval()`.

Custom Example:

```
function processUserInput(callback) {
  const name = "Jagdeep";
  // HOF calls the callback function
  console.log(callback(name));
}
function greet(name) { return "Hello " + name; }

processUserInput(greet); // Output: Hello Jagdeep
```

Q4. What are generators in JavaScript? How do they differ from regular functions?

Generators are special functions that can be exited and later re-entered, preserving the context (variable bindings) across re-entrances. They are defined using an asterisk (`function*`).

They use the **`yield` keyword**, which pauses the function and returns the value next to it. Calling `.next()` on the generator object resumes execution.

Difference from Regular Functions:

Feature	Regular Function	
Control Flow	Runs to completion once executed.	**Can pause and resume** multiple times.
Return Value	Returns a single value.	Returns a sequence of values (an Iterator).

Example:

```
function* numberGen() {
  yield 1;
  yield 2;
}
const gen = numberGen();
console.log(gen.next().value); // 1 (Pauses)
console.log(gen.next().value); // 2 (Resumes and pauses)
```

Q5. What is the 'call', 'apply', and 'bind' method? How do they work?

These are methods available on all functions that allow you to explicitly set the value of the **`this` keyword** for a function call.

Method	Purpose	Argument Style
call()	Invokes the function immediately with a specified `this` context.	**Comma-separated** arguments (`func.call(obj, arg1, arg2)`)
apply()	Invokes the function immediately with a specified `this` context.	**Arguments as an array** (`func.apply(obj, [arg1, arg2])`)

Method	Purpose	Argument Style
bind()	**Returns a new function** with `this` bound permanently to the provided object, but does not invoke it immediately.	Comma-separated arguments (`const bound = func.bind(obj, arg1)`)

Q6. Explain the concept of memoization in JavaScript.

Memoization is an optimization technique used to **cache the results of expensive function calls** and return the cached result when the same inputs occur again, thereby avoiding redundant computations.

It's essentially trading memory space for execution time.

Example:

```
function memoizedAdd() {
  const cache = {};
  return function(n) {
    if (cache[n]) {
      return cache[n]; // Return from cache
    }
    console.log("Calculating..."); // Executes only if input is new
    cache[n] = n + 10;
    return cache[n];
  };
}
const add = memoizedAdd();
console.log(add(5)); // Calculating... (15)
console.log(add(5)); // (15) - Returns from cache
```

Q7. What is the difference between deep and shallow copy in JavaScript?

This distinction is crucial when copying objects or arrays, especially those with nested structures.

Copy Type	Description	Example Technique
Shallow Copy	Copies **only the first level** of properties. Nested	Spread operator (`{...obj}`), `Object.assign()`.

Copy Type	Description	Example Technique
	objects still share references to the original data, so modifying the copy affects the original.	
Deep Copy	Copies **all levels of properties**. No shared references remain, so modifying the copy does not affect the original object.	`JSON.parse(JSON.stringify(obj))` (simplest for non-function data), or dedicated libraries like Lodash `_.cloneDeep()`.

Shallow Copy Example Issue:

```
const original = { a: 1, b: { c: 2 } };
const shallow = { ...original }; // Shallow Copy

shallow.b.c = 99; // Change nested object in copy
console.log(original.b.c); // Output: 99 (Original was affected!)
```

7. Error Handling

Q1. How does error handling work in JavaScript? What are try-catch blocks?

JavaScript uses the **`try-catch` statement** to handle runtime errors gracefully, allowing the program to continue execution instead of crashing.

- **try**: Contains the block of code that might throw an error.
- **catch**: Contains the code that is executed if an error occurs in the `try` block, receiving the error object as an argument.
- **finally** (optional): Contains code that is executed **regardless** of whether an error occurred or was caught.

Example:

```
try {
  let result = 10 / x; // Throws ReferenceError because 'x' is undefined
```

```
} catch (error) {
  console.log("Error occurred:", error.message); // Handles the error
} finally {
  console.log("Execution completed"); // Always runs
}
```

Q2. What is the difference between 'throw' and 'return' in error handling?

- **throw**: Used to **generate a custom error intentionally**. It immediately **stops the function execution** and transfers control to the nearest `catch` block (or halts the program if none is found). It signals an exceptional situation.
- **return**: Used to **send a value back from a function**. It stops function execution but is part of the normal program flow and **does not signal an error**.

Example:

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero"); // Stops execution, signals
  }
  return a / b; // Returns value, normal flow
}
```

Q3. Explain the concept of custom errors in JavaScript.

Custom Errors are user-defined error types that allow developers to handle specific, non-standard error conditions more accurately than relying only on built-in error types (like `TypeError` or `ReferenceError`).

They are typically created by **extending the built-in `Error` class**, which ensures they inherit essential properties like `message` and `stack`.

Example:

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError"; // Custom error name
  }
```

```
}
try {
  throw new ValidationError("Invalid input data");
} catch (error) {
  console.log(error.name);    // ValidationError
  console.log(error.message); // Invalid input data
}
```

---

8. Miscellaneous

Q1. What is 'strict mode' in JavaScript? What are the benefits of using it?

Strict Mode (activated by the directive `'use strict';`) is a feature introduced in ES5 that enforces stricter parsing and error handling behavior in your JavaScript code.

Benefits:

- **Prevents accidental global variables** (must declare variables with `var`, `let`, or `const`).
- Disallows duplicate parameter names in functions.
- Makes `this` value **`undefined`** **in functions** that aren't object methods, preventing accidental access to the global object.
- Catches silent errors (like assigning to read-only properties) and throws exceptions instead.

It can be applied globally at the top of a script or locally at the top of a function.

Q2. What is the purpose of the 'eval' function? Why is it discouraged?

The **`eval()`** **function** executes a string of JavaScript code passed to it dynamically at runtime.

Purpose Example:

```
eval("console.log(2 + 2)"); // 4
```

Why It Is Discouraged:

- **Security Risks**: It can execute arbitrary code passed from an external source, posing a severe **security vulnerability** (Injection attacks).

- **Performance Reduction**: Code executed via `eval()` cannot be fully optimized by JavaScript engines, as the execution environment is unpredictable.
- **Debugging Difficulty**: Code inside `eval()` can be hard to trace and debug.

Q3. What is JSON, and how do you convert a JavaScript object into JSON?

JSON (JavaScript Object Notation) is a lightweight, text-based data format used primarily for **storing and exchanging data** between a server and a web application.

It is language-independent but uses conventions that are familiar to programmers of the C-family of languages, including JavaScript.

Conversion Methods (using the global `JSON` object):

- **Object → JSON String**: Use `JSON.stringify(object)`.
- **JSON String → JavaScript Object**: Use `JSON.parse(jsonString)`.

Example:

```
const obj = { name: "Jagdeep", age: 22 };
const json = JSON.stringify(obj);
console.log(json); // {"name":"Jagdeep","age":22}

const parsed = JSON.parse(json);
console.log(parsed.name); // Jagdeep
```

Q4. How does garbage collection work in JavaScript?

JavaScript uses **automatic garbage collection (GC)**. This mechanism is managed by the JavaScript engine to automatically free up memory occupied by objects that are **no longer "reachable"** by the code.

The most common algorithm used is **Mark-and-Sweep**:

1. **Marking**: The garbage collector traverses the object graph starting from "roots" (global variables, currently executing functions) and marks all reachable objects as active.
2. **Sweeping**: The collector then iterates through the heap and **removes (sweeps) all objects that were not marked**, thereby reclaiming the memory.

Example:

```
let user = { name: "John" };
user = null; // The original object is now unreachable and eligible for
```

Q5. Explain debouncing and throttling in JavaScript.

Both techniques are used to **limit the rate at which a function is executed**, typically in response to rapid user input or events like scrolling/resizing.

- **Debouncing**: Ensures the function is called **only after a specified period of inactivity** (the delay). If the event fires again before the delay is over, the timer is reset.
  - **Use Case**: Search bar input (run the API call only when the user stops typing).
- **Throttling**: Ensures the function is called **at most once within a given time period** (the limit). It guarantees a consistent execution rate.
  - **Use Case**: Handling continuous events like scrolling or mouse movements.

Q6. What is a polyfill in JavaScript?

A **Polyfill** is a piece of JavaScript code (or library) that provides the functionality of a **newer web standard or feature to older environments (browsers)** that lack native support for it.

The goal is to allow developers to use modern JavaScript APIs without sacrificing backward compatibility.

Example:

Polyfilling `Array.prototype.includes` for older browsers:

```
if (!Array.prototype.includes) {
  // If the feature is missing, define it:
  Array.prototype.includes = function(value) {
    return this.indexOf(value) !== -1;
  };
}
```

© 2025 Kaka Sharma.